



|                                                                                                                          |                          |                    |                   |
|--------------------------------------------------------------------------------------------------------------------------|--------------------------|--------------------|-------------------|
| <b>Lab Report No : 6</b>                                                                                                 |                          |                    |                   |
| <b>Lab Report Name : Adversarial Search: Minimax Algorithm (Tic-Tac-Toe), Alpha-Beta Pruning Algorithm (Tic-Tac-Toe)</b> |                          |                    |                   |
| <b>Course Title : Artificial Intelligence Sessional</b>                                                                  |                          |                    |                   |
| <b>Course Code : CSE 422</b>                                                                                             |                          |                    |                   |
| <b>Name : Md. Rakibul Hassan</b>                                                                                         |                          |                    |                   |
| <b>ID : 0822220105101024</b>                                                                                             |                          |                    |                   |
| <b>Level : 4</b>                                                                                                         | <b>Term : 2</b>          | <b>Section : A</b> | <b>Group : G1</b> |
| <b>Date of Submission : 19.07.2026</b>                                                                                   | <b>Semester : Spring</b> | <b>Year : 2026</b> |                   |

### Marking Rubric

| Problem Understanding & Report Clarity (3) | Implementation (4) | Results, Analysis & Presentation (3) | Total (10) |
|--------------------------------------------|--------------------|--------------------------------------|------------|
|                                            |                    |                                      |            |

### Adversarial Search: Minimax Algorithm (Tic-Tac-Toe):

#### Key Learnings:

- Learned that Minimax guarantees the optimal move when both players play perfectly.
- Learned that the algorithm explores every possible future game state, making it suitable for small games like Tic-Tac-Toe.
- Learned that the evaluation score propagates back through the recursion to determine the best current move.

#### Code Implementation:

```
[1]
✓ 0s
import math

def print_board(board):
    for row in [board[i:i+3] for i in range(0, 9, 3)]:
        print(row)

def check_winner(board):
    win_combinations = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]

    for a, b, c in win_combinations:
        if board[a] == board[b] == board[c] and board[a] != " ":
            return board[a]

    if " " not in board:
        return "Draw"

    return None
```

Figure 1: Helper functions for board display and winner detection

```

[1]
✓ Os
def minimax(board, is_maximizing):
    winner = check_winner(board)

    if winner == "X":
        return -1
    if winner == "O":
        return 1
    if winner == "Draw":
        return 0

    if is_maximizing:
        best_score = -math.inf

        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                score = minimax(board, False)
                board[i] = " "
                best_score = max(best_score, score)

        return best_score

    else:
        best_score = math.inf

```

Figure 2: Implementation of the Minimax algorithm for evaluating optimal game moves. (First-Half)

```

[1]
✓ Os
    for i in range(9):
        if board[i] == " ":
            board[i] = "X"
            score = minimax(board, True)
            board[i] = " "
            best_score = min(best_score, score)

    return best_score

```

Figure 3: Implementation of the Minimax algorithm for evaluating optimal game moves.

```

[1]
✓ Os
def best_move(board):
    best_score = -math.inf
    move = None

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = minimax(board, False)
            board[i] = " "

            if score > best_score:
                best_score = score
                move = i

    return move

board = [" "] * 9
board[0] = "X"

print_board(board)
print("AI chooses position:", best_move(board))

```

Figure 4: AI move selection using the Minimax algorithm and program execution.

### **Input-Output:**

```
...  ['X', ' ', ' ', ' ']  
      [' ', ' ', ' ', ' ']  
      [' ', ' ', ' ', ' ']  
      AI chooses position: 4
```

---

*Figure 5: Sample Input-Output*

### **Result Analysis:**

- The Minimax algorithm successfully selected the optimal move for the AI by evaluating all possible future game states.
- The AI guaranteed the best possible outcome, assuming both players made optimal decisions.
- The recursive search correctly identified winning, losing, and draw situations before selecting a move.
- Although the algorithm produced the correct result, it explored every possible game state, making it computationally expensive.

## Adversarial Search: Minimax Algorithm (Tic-Tac-Toe):

### Key Learnings:

- Learned that Alpha-Beta Pruning reduces the number of nodes explored without affecting the final decision.
- Learned that pruning occurs when  $\beta \leq \alpha$ , allowing the algorithm to skip branches that cannot improve the outcome.
- Learned that Alpha-Beta Pruning is more efficient than Minimax, especially for games with large search trees, because it saves both computation time and memory.

### Code Implementation:

```
[2]
✓ Os
import math

def print_board(board):
    for row in [board[i:i+3] for i in range(0, 9, 3)]:
        print(row)

def check_winner(board):
    win_combinations = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]

    for a, b, c in win_combinations:
        if board[a] == board[b] == board[c] and board[a] != " ":
            return board[a]

    if " " not in board:
        return "Draw"

    return None
```

Figure 1: Helper functions for board display and winner detection

```
[2]
✓ Os
def alpha_beta(board, depth, alpha, beta, is_maximizing):
    winner = check_winner(board)

    if winner == "X":
        return -1
    if winner == "O":
        return 1
    if winner == "Draw":
        return 0

    if is_maximizing:
        max_eval = -math.inf

        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                eval = alpha_beta(board, depth + 1, alpha, beta, False)
                board[i] = " "
                max_eval = max(max_eval, eval)
                alpha = max(alpha, eval)

                if beta <= alpha:
                    break

        return max_eval
```

Figure 2: Optimizing game tree search in `alpha_beta()` function (First Half)

```

[2]
✓ 0s
else:
    min_eval = math.inf

    for i in range(9):
        if board[i] == " ":
            board[i] = "X"
            eval = alpha_beta(board, depth + 1, alpha, beta, True)
            board[i] = " "
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)

            if beta <= alpha:
                break

    return min_eval

```

Figure 3: Optimizing game tree search in `alpha_beta()` function (Second Half)

```

[2]
✓ 0s
def best_move(board):
    best_score = -math.inf
    move = None

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = alpha_beta(board, 0, -math.inf, math.inf, False)
            board[i] = " "

            if score > best_score:
                best_score = score
                move = i

    return move

board = [" "] * 9
board[3] = "X"

print_board(board)
print("AI chooses position:", best_move(board))

```

Figure 4: AI move selection using Alpha-Beta Pruning and program execution.

### Input-Output:

```

...  [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
      ['X', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
      [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
      AI chooses position: 0

```

Figure 5: Sample Input-Output

### Result Analysis:

- The alpha-beta pruning algorithm selected the same optimal move as the minimax algorithm.
- It reduced the number of game states explored by pruning unnecessary branches using the alpha and beta values.
- The pruning process improved execution efficiency without affecting the correctness of the final decision.
- Compared to Minimax, Alpha-Beta Pruning required fewer computations, making it more suitable for larger game search problems.